Ideation Phase

Implement Client Script & UI Policy (Incident)

Project Name: Implement Client Script & UI Policy (Incident) | Team ID: Group 03 | Date: 01 October 2026

Project Context

Incident records require consistent and accurate data entry to support effective triage, routing and resolution. Relying only on user awareness and manual checks can lead to incomplete, inconsistent or incorrect submissions, affecting reporting accuracy, SLA compliance and overall service quality. This project enforces conditional field behavior and validation directly at the user interface level using ServiceNow UI Policies, UI Policy Actions and Client Scripts, so Incident records meet defined business rules before they are submitted.

Objective

Demonstrate how ServiceNow client-side controls enforce data integrity on Incident records by dynamically making fields mandatory, auto-populating values, controlling field behavior and preventing record submission when required conditions are not met, so Incidents are created with complete and valid information.

Primary Stakeholders

Caller / Service Desk Agent: needs a guided Incident form that shows relevant fields and flags missing information immediately.
Resolver / Fulfiller Group: needs complete, correctly categorized Incidents for accurate routing and faster resolution.
Process Owner / ServiceNow Admin: needs enforceable, maintainable rules that protect reporting accuracy and SLA compliance.

Skills

Incident Management | UI Policy | UI Policy Actions | Client Scripts | Form Validation

Team Members

Name	Role
Anirudh	Team Lead
Mohan Raj	Team Member
Shyam Sundhar	Team Member

Empathize & Discover

Empathy Map Canvas

The maps below capture the behaviours, needs and challenges of the three main stakeholder groups of the Incident form.

1. Caller / Service Desk Agent

SAYS	HEARS / SEES
“Which fields do I have to fill in?” “Why was my Incident sent back?” “What information is required to submit?”	Long forms with many fields Reminders about missing details Feedback on incomplete tickets
PAINS	GAINS / NEEDS
Irrelevant fields clutter the form Errors found only after submission Unsure of required values	Only relevant fields shown Instant, clear validation messages Sensible auto-filled defaults

2. Resolver / Fulfiller Group

SAYS	HEARS / SEES
“The description is too vague to act on.” “This was routed to the wrong group.” “Where are the resolution notes?”	Incidents missing category or details Reassignment requests Requests for more information
PAINS	GAINS / NEEDS
Incomplete records slow triage Time lost chasing the caller Resolved tickets without notes	Complete, consistent Incident data Accurate routing Mandatory resolution details

3. Process Owner / ServiceNow Admin

SAYS	HEARS / SEES
“Can we trust our Incident reports?” “How do we enforce the rules consistently?”	SLA breaches linked to poor data Inconsistent values in reports Manual data-quality checks
PAINS	GAINS / NEEDS
Reliance on user awareness alone Inaccurate reporting Scattered, hard-to-maintain rules	Rules enforced at the UI level Reliable reports and SLA tracking Standardized, reusable controls

Define the Problem Statements

Customer Problem Statement Template

These statements focus on the user point of view before defining the solution.

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Caller / Service Desk Agent	create an Incident quickly with the right details	the form shows every field and I learn about missing data only after submitting	there is no guidance or conditional behavior on the form	frustrated and unsure what information is required
PS-2	Resolver / Fulfiller Group	triage, route and resolve Incidents efficiently	records arrive incomplete, inconsistently categorized or without resolution notes	data entry depends on manual checks and user awareness	slowed down and forced to chase missing information
PS-3	Process Owner / ServiceNow Admin	ensure accurate reporting and SLA compliance	invalid or incomplete data still gets submitted	business rules are not enforced at the point of entry	concerned about data quality and service-level risk

Core Problem

Incident data quality depends too heavily on user awareness. Without enforced conditional field behavior and save-time validation, incomplete or incorrect records reach the queue, harming triage, routing, SLA compliance and reporting. Callers need guided forms, resolvers need complete records, and process owners need rules enforced automatically at the user interface level.

Brainstorm & Idea Prioritization Template

Step 1: Team Gathering, Collaboration and Select the Problem Statement

How might we enforce consistent, valid Incident data at the point of entry so that every record is complete and correct before it is submitted?

Step 2: Brainstorm, Idea Listing and Grouping

Idea / Group	Concept
Conditional Mandatory Fields	UI Policy actions make fields mandatory based on conditions, e.g. resolution code and close notes when State is Resolved/Closed, or on-hold reason when State is On Hold.
Dynamic Field Visibility	UI Policy actions show or hide fields so users only see what is relevant to the current state.
Read-only Field Control	Lock fields that should not be edited after certain conditions to protect data integrity.
Auto-populate Values	Client Script (onChange) fills dependent fields or defaults when a key field such as Category or Caller changes.
Save-time Validation	Client Script (onSubmit) checks business rules and blocks submission with a clear error message.
User Guidance Messages	Field messages and form info/error messages explain what is wrong and how to fix it.
UI Policy vs Client Script Guide	Use UI Policy for declarative field behavior; use Client Script when logic or validation needs scripting.
Test & Verification Scenarios	Positive and negative test cases confirming fields behave correctly and invalid records cannot be saved.

Step 3: Idea Prioritization

The ideas are compared using expected user value and implementation effort. The proposed concept integrates the core ideas into one Incident form implementation.

Idea	User Value	Implementation Effort	Role
Conditional Mandatory Fields	High	Low	Core
Dynamic Field Visibility	High	Low	Core
Save-time Validation (onSubmit)	High	Medium	Core
Auto-populate Values (onChange)	Medium–High	Medium	Core
Read-only Field Control	Medium	Low	Supporting
User Guidance Messages	Medium	Low	Supporting
Test & Verification Scenarios	High	Low–Medium	Supporting
UI Policy vs Client Script Guide	Medium	Low	Supporting

Proposed Concept

Incident Form Data-Integrity Controls — a ServiceNow implementation of UI Policies, UI Policy Actions and Client Scripts. The solution brings together conditional mandatory fields, dynamic field visibility, read-only control, auto-populated values, save-time validation and user guidance messages, so only complete and valid Incidents are submitted.

Expected Outcome

Open Incident → Apply UI Policy → Auto-populate → Validate on Submit → Save Clean Record → Accurate Triage → Reliable SLA & Reporting.

Template References

Empathy Map Canvas: Mural template. Customer Problem Statement: Miro template. Brainstorm & Idea Prioritization: Mural template. The template structure has been adapted to this project.